

MIDTERM PHASE 0 - HUMAN DETECTOR AND TRACKER

1. Overview

This document outlines the plan for the design and development of a new **Human Obstacle Detector and Tracker** module for Acme Robotics. The purpose of this module is to identify the 3D location of one or more humans in a robot's environment using a single monocular video camera.

This component is critical for safe human-robot interaction and navigation. It will provide downstream components, such as a motion planner, with a list of detected human obstacles and their positions relative to the robot. This allows the robot to implement behaviors like stopping, slowing down, or navigating around people.

The module will interface with the robot's camera system by accepting raw video frames. It will output a structured data list of DetectedObjects, where each object contains a unique tracking ID and its (x, y, z) position in the robot's reference frame.

Core Assumptions:

1. The module receives a continuous video stream from a single, calibrated monocular camera or a clear well lit image.
2. The robot's control system can provide the camera's mounting parameters (height from the ground, pitch angle).
3. Humans are walking on a flat, continuous ground plane.
4. Occlusion handling (i.e., tracking a person after they go behind another object) is not required for this phase.

2. Reference Materials

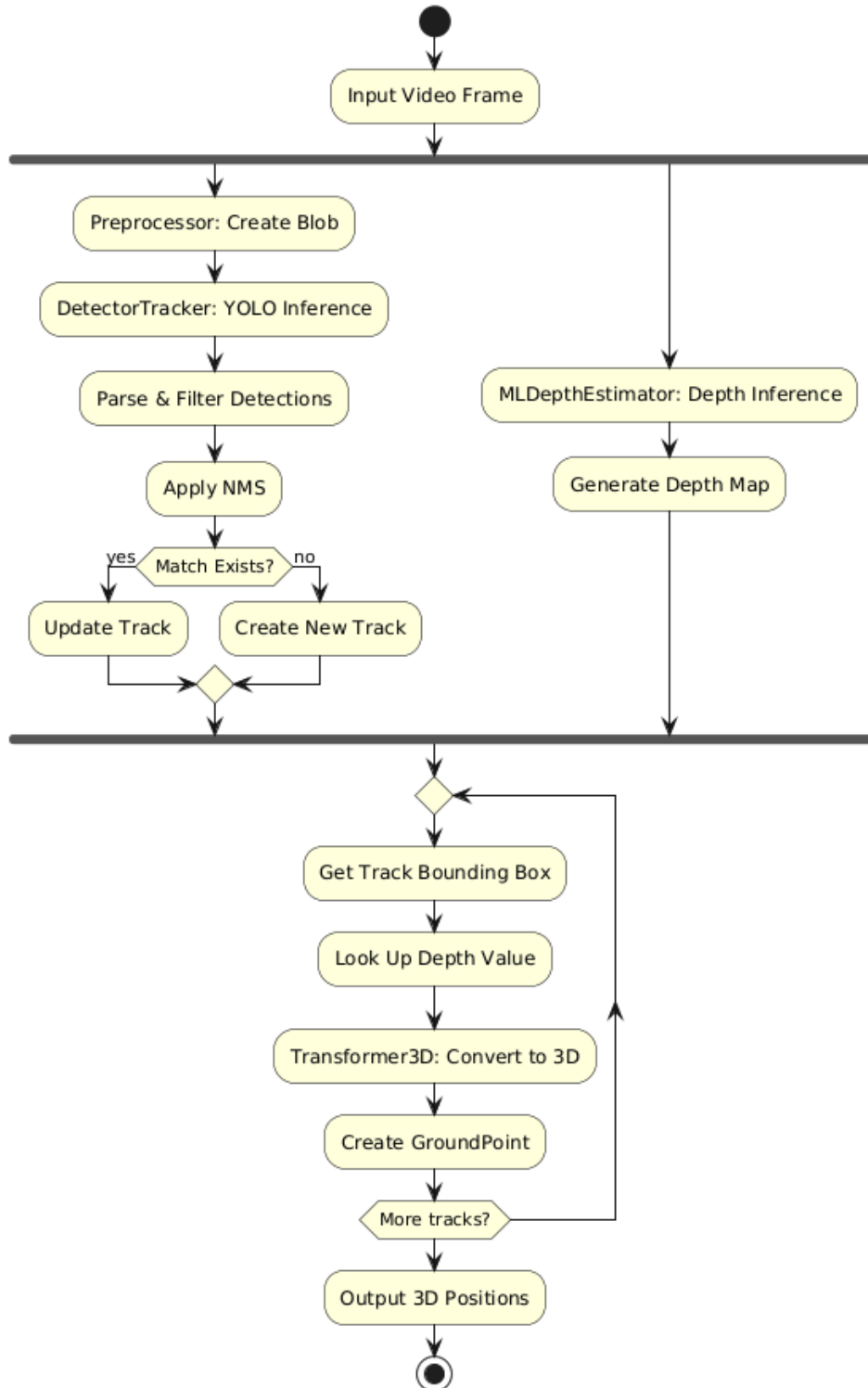
- *Software Engineering: The Current Practice* by Václav Rajlich (provided textbook)
- Project Assignment Brief (Acme Robotics Mid-Term Project)
- Google C++ Style Guide
- OpenCV Library Documentation

3. Definitions and Acronyms

- **AIP:** Agile Iterative Process
- **CI:** Continuous Integration
- **TDD:** Test-Driven Development
- **UML:** Unified Modeling Language
- **DNN:** Deep Neural Network
- **Monocular Camera:** A single camera, which provides 2D images without inherent depth information.
- **Robot Reference Frame (Pose):** A 3D coordinate system centered on the robot.

4. System Architecture and Interfaces

Activity Diagram: Human Detection and Tracking System



Interface Assumptions:

1. Camera provides calibrated images at least 10 FPS
2. Camera intrinsics and extrinsics are known and stable
3. Ground plane is flat and continuous (no stairs or ramps)
4. Humans are visible and not fully occluded
5. Adequate lighting for camera operation (>50 lux)

5. Process

We will follow the **Agile Iterative Process (AIP)** as described in Chapter 13 of the textbook , combined with a **Test-Driven Development (TDD)** methodology.

1. **Phase 0 (Week 1):** Planning & Design.
2. **Phase 1 (Week 2):** Sprints 1 & 2.
 - **Sprint 1:** Implement core classes, camera interface, and 2D detection (TDD).
 - **Sprint 2:** Implement 2D tracker and 3D location estimator (TDD).
3. **Phase 2 (Week 3):** Sprint 3 & Final Delivery.
 - **Sprint 3:** Integration, system testing, documentation, and code coverage push (to 90%+).

Our development will follow the TDD cycle:

1. Write a failing unit test for a new feature (Red).
2. Write the minimal code to make the test pass (Green).
3. Refactor the code while ensuring all tests still pass (Refactor).

Quality will be ensured via GitHub CI running cppcheck (static analysis), Google Test (unit tests), and CodeCov (coverage monitoring) on every commit.

6. Organization

We are a two-person team. We will execute **Pair Programming** for all development tasks.

- **Driver:** The developer writing the code.
- **Navigator:** The developer reviewing the code in real-time, considering strategic design, and identifying potential bugs or test cases.
- **Rotation:** We will switch roles for every Different Phase for the assignment.
- **Documentation:** Our pair programming sessions will be documented in our Github repository.

7. Technologies

- **Language:** C++17
- **Build System:** CMake
- **Testing:** Google Test (gtest)
- **Static Analysis:** cppcheck
- **Documentation:** Doxygen
- **CI/CD:** GitHub Actions
- **Coverage:** CodeCov
- **External Libraries:**
 - **OpenCV :** Used for all image processing, video I/O, and for loading and running the pre-trained DNN model for 2D detection.

8. Algorithm

This section details the key algorithms used in the module.

8.1 YOLO Object Detection

We use YOLOv8-nano, a state-of-the-art real-time object detection model:

- Training: Pre-trained on COCO dataset (80 classes, we use "person" class only)
- Input: 640×640 RGB image, normalized to [0, 1]
- Output: Bounding boxes with class probabilities and confidence scores
- Post-processing: Non-Maximum Suppression (NMS) with IoU threshold 0.45

8.2 IoU-Based Multi-Object Tracking

A simple and effective tracking algorithm for maintaining persistent IDs:

Algorithm:

For each new frame:

1. Get detections from YOLO (bounding boxes)
2. For each existing track:
 - Calculate IoU with all current detections
 - If max IoU > threshold (e.g., 0.3), associate detection to track
 - Update track position and age
3. For unassociated detections:
 - Create new track with new unique ID
4. For unassociated tracks:
 - Increment time_since_update
 - Delete if not updated for N frames (e.g., N=5)
5. Return list of active tracks

Advantages:

- Simple and fast (no complex optimization)
- Works well for non-occluded scenarios
- No training required

8.3 Monocular Depth Estimation

We use deep learning-based monocular depth estimation:

- Approach: MiDaS (Mixed Data Sampling) or Depth Anything networks
- Input: RGB image (resized to 256×256 or 384×384)
- Output: Relative depth map
- Key Insight: Monocular depth is inherently ambiguous (scale ambiguity). The network predicts *relative* depth, not metric depth. We mitigate this by assuming a flat ground plane and using the known camera height as a reference.

8.4 3D Coordinate Transformation

Converts 2D pixel coordinates to 3D robot coordinates using the pinhole camera model:

- Step 1: Pixel to Camera Coordinates Given pixel (u, v) and depth d:
 $X_{\text{camera}} = (u - c_x) \times d / f_x$ $Y_{\text{camera}} = (v - c_y) \times d / f_y$ $Z_{\text{camera}} = d$
Where (fx, fy) are focal lengths and (cx, cy) is the principal point.
- Step 2: Camera to Robot Coordinates Apply the 4x4 homogeneous transformation matrix ($T_{\text{camera_to_robot}}$) that encodes the camera's rotation (R) and translation (t) relative to the robot's origin.
$$\begin{bmatrix} X_{\text{robot}} \\ Y_{\text{robot}} \\ Z_{\text{robot}} \\ 1 \end{bmatrix} = \begin{bmatrix} R_{11} & R_{12} & R_{13} & t_x \\ R_{21} & R_{22} & R_{23} & t_y \\ R_{31} & R_{32} & R_{33} & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_{\text{camera}} \\ Y_{\text{camera}} \\ Z_{\text{camera}} \\ 1 \end{bmatrix}$$

This transforms camera coordinates ($X_{\text{camera}}, Y_{\text{camera}}, Z_{\text{camera}}$) into the robot-centric reference frame ($X_{\text{robot}}, Y_{\text{robot}}, Z_{\text{robot}}$).

9. Management

Task Management: We will use GitHub Issues to manage our product backlog and sprint tasks.

- **Version Control:** We will use a main branch for stable releases and feature branches for all new development. All feature branches will require a passing CI build and a review from the other team member before merging into main.
- **Risks & Mitigations:**
 1. **Risk:** Inaccurate 3D location from monocular data.
 - **Mitigation:** This is the project's primary technical challenge. We will mitigate it by clearly defining our assumption (flat ground plane) and implementing a well-documented geometric projection algorithm. Accuracy will be dependent on precise camera calibration, which we will simulate in our unit tests.
 2. **Risk:** Real-time performance of the DNN model.
 - **Mitigation:** We will select a lightweight, high-performance model (YOLO v8) designed for edge devices.
 3. **Risk:** TDD for computer vision is difficult.
 - **Mitigation:** We will test known inputs. For example, we will save a sample image with known human locations and write a test asserting that our IDetector finds them. We will test the ILocationEstimator by feeding it exact 2D pixel coordinates and asserting the correct 3D output coordinates.

10. Cost

As this is a 3-week academic project, the "cost" is defined by development time.

- **Team:** 2 developers.
- **Project Duration:** 3 weeks.
- **Total Effort (Estimate):** 3 weeks * 2 developers * 20 hours/week = 120 person-hours.
- **Software Cost:** \$0. All development tools and libraries (C++17, CMake, GTest, OpenCV) are free and open-source.